



**MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA**

**Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software and Automation Engineering**

VREMERE ADRIAN, FAF-232

Report

Laboratory Work n.4.1

Binary Actuator Control with Signal Conditioning

on Embedded Systems

Checked by:

Martîniuc Alexei, *univ. asist.*
FCIM, UTM

Chişinău – 2026

1. Domain Analysis

1.1. Objective of the Laboratory Work

The objective of this laboratory work is to design and implement a binary actuator control system on the Arduino Mega 2560 microcontroller that accepts commands from the user via a serial terminal, applies a multi-stage signal conditioning pipeline to eliminate spurious transitions, and drives a physical output while continuously reporting the system state on both an LCD display and the serial monitor.

The core learning objectives are to understand the concepts of binary actuator control, software-based signal conditioning for discrete signals (saturation, debouncing, and persistent state validation), and preemptive task scheduling with FreeRTOS. The lab also reinforces the principle of separating concerns across modular software layers, from raw input parsing through signal processing to hardware actuation and reporting.

1.2. Problem Definition

The task, as specified by the laboratory instructor, is reproduced below:

Control Actuator Binar

Pentru un actuator binar (ex.: bec de iluminare prin releu sau alt actuator convenit cu mentorul), se va realiza o aplicație pentru MCU care:

- *interpretează comenzi de control de la utilizator prin STDIO (serial sau keypad)*
- *controlează actuatorul*
- *afișează starea curentă pe un display LCD*

Aplicația va implementa:

- **Control actuator binar:** Primește comenzi ON/OFF prin STDIO. Aplică debouncing. Controlează starea la o recurență configurabilă (50–100 ms). Expune starea printr-o interfață internă.
- **Condiționare semnal:** Aplică saturare, debouncing software, și validare persistentă a stării.
- **Afișare și raportare:** Afișează pe LCD sau STDIO un raport structurat la 500 ms.

The system therefore requires three distinct processing stages. The user sends the string ON or OFF via the serial terminal. That raw command passes through a conditioning pipeline that first saturates the value to a strict boolean, then requires the value to remain stable for a minimum debounce window, and finally confirms it with a minimum number of consecutive identical samples before the hardware output is actually toggled. The conditioned state is displayed on the LCD and reported periodically on the serial monitor.

1.3. Used Technologies

UART Serial Communication and STDIO Redirection

UART (Universal Asynchronous Receiver-Transmitter) is the hardware serial interface used to connect the Arduino Mega 2560 to the host PC. The microcontroller's UART0 peripheral is connected to a USB-to-serial converter, making it accessible as a virtual COM port on the host. The application configures the interface at 9600 baud with the standard 8N1 framing (8 data bits, no parity, 1 stop bit).

Rather than using the Arduino-specific `Serial.print()` API at the application level, the `StdioSerial` library redirects the C standard I/O streams — `stdout` and `stdin` — to the hardware serial port using the AVR libc `fdev_setup_stream()` mechanism. This allows all formatted output to use the portable `printf()` function and all input to be read through `Serial.available()` / `Serial.read()`, keeping the application layer independent of hardware-specific serial APIs.

Binary Actuator Control and Signal Conditioning

A binary actuator is any device that operates in exactly two discrete states: active (ON) and inactive (OFF). Common examples include relays driving mains loads, solenoid valves, indicator LEDs, and buzzers. In this laboratory the actuator is simulated by a red LED connected to a current-limiting resistor on pin D7.

Control of a binary actuator from a user interface introduces the risk of spurious state changes caused by rapid repeated commands, communication noise, or accidental double-presses. A signal conditioning pipeline addresses this by introducing three filtering stages. Saturation maps any raw input value to a strict element of the set `{false, true}`. Software debouncing prevents the actuator state from changing unless the input has been stable for a minimum duration; this mirrors hardware debouncing used for mechanical pushbuttons and relay contacts. Persistent state validation requires the input to be confirmed by a minimum number of consecutive identical samples, providing an additional layer of filtering against transient glitches.

FreeRTOS Preemptive Task Scheduler

FreeRTOS is a real-time operating system kernel that provides preemptive multitasking on microcontrollers. On the Arduino Mega 2560, the `feilipu/FreeRTOS` library ports the kernel to the AVR ATmega2560 architecture with a default tick rate of 200 Hz. Each task has its own stack, a priority level, and runs a while-forever loop using `vTaskDelayUntil()` for accurate period control.

Preemptive scheduling ensures that higher-priority tasks (input reading at priority 3) can interrupt lower-priority ones (display at priority 1), guaranteeing responsive user interaction regardless of display rendering time. Shared state between tasks is protected by FreeRTOS mutexes to prevent race conditions.

I2C Protocol and LCD Display

The Inter-Integrated Circuit (I2C) protocol is a two-wire synchronous serial bus that connects the microcontroller to the LCD module through a PCF8574 I/O expander. The Arduino Mega 2560 provides hardware I2C on pins SDA (D20) and SCL (D21). The standard 7-bit I2C address of 0x27 is used for the LCD backpack module. The `LcdDisplay` library wraps the `LiquidCrystal_I2C` driver, providing a clean line-based API that automatically pads shorter strings to clear residual characters.

1.4. Hardware Components

Arduino Mega 2560

The Arduino Mega 2560 is a microcontroller development board built around the 8-bit AVR ATmega2560 running at 16 MHz. It provides 256 KB of Flash program memory, 8 KB of SRAM for runtime data, 54 digital I/O pins, and 16 analog inputs. Four hardware UART ports enable simultaneous serial communication with the host PC and other peripherals. The on-board USB-to-serial converter allows programming and monitoring over a standard USB cable. In this laboratory the board acts as the system's central processing unit, executing the FreeRTOS kernel and all three application tasks.

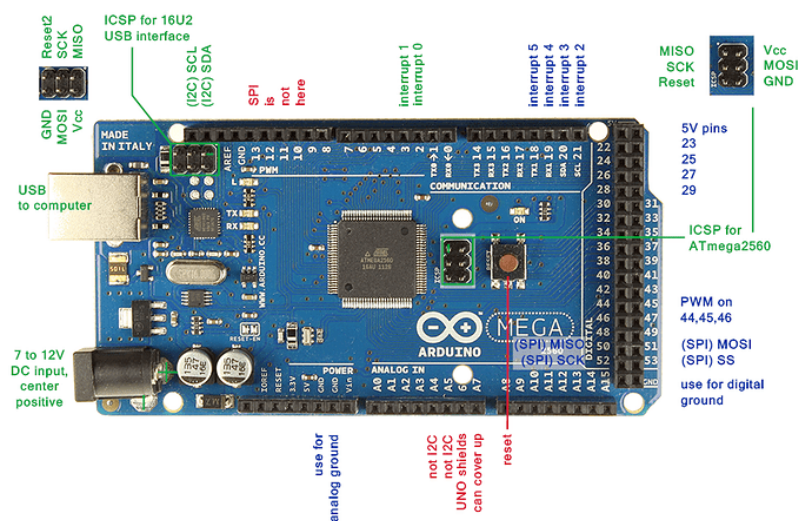


Figure 1.1 – Arduino Mega 2560 development board

LED and Current-Limiting Resistor

A standard 5 mm red LED is used to simulate the binary actuator output. An LED requires a current-limiting resistor in series to prevent excessive current from damaging the device. With a supply of 5 V, a typical LED forward voltage of 2.0 V, and a desired current of approximately 14 mA, the required resistance is $(5.0 - 2.0)/0.014 \approx 214\Omega$, so a standard 220 Ω resistor is used. In a real-world application the LED and resistor would be replaced by a relay module driving a

mains load; the software interface and signal conditioning pipeline remain identical regardless of the actual load type.

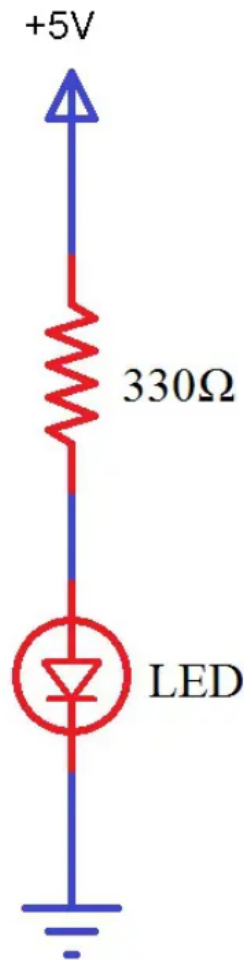


Figure 1.2 – 5 mm red LED (simulates binary actuator output)

1.5. Software Components

PlatformIO

PlatformIO is an open-source embedded development ecosystem integrated into VS Code. It manages board configurations, toolchain installation, library dependency resolution, and project building. Each lab is defined as a named environment in `platformio.ini` with its own build flags, source filter, and library list. Switching between labs requires only selecting a different environment in the IDE — no source files need to be modified. The `pio run -e lab4_1` command compiles the firmware for this lab.

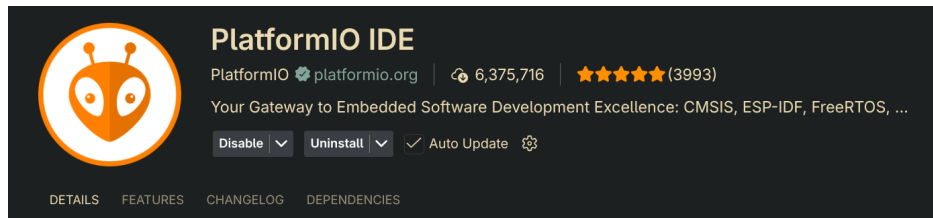


Figure 1.3 – PlatformIO IDE in VS Code with Lab 4.1 environment

Wokwi

Wokwi is a browser-based and VS Code-integrated simulator for embedded systems that supports the Arduino Mega 2560. It loads the compiled PlatformIO firmware directly from `.pio/build/lab4`, simulating both the microcontroller execution and the virtual serial terminal. This allows functional testing — sending ON/OFF commands and observing the LED and LCD behaviour — without physical hardware.

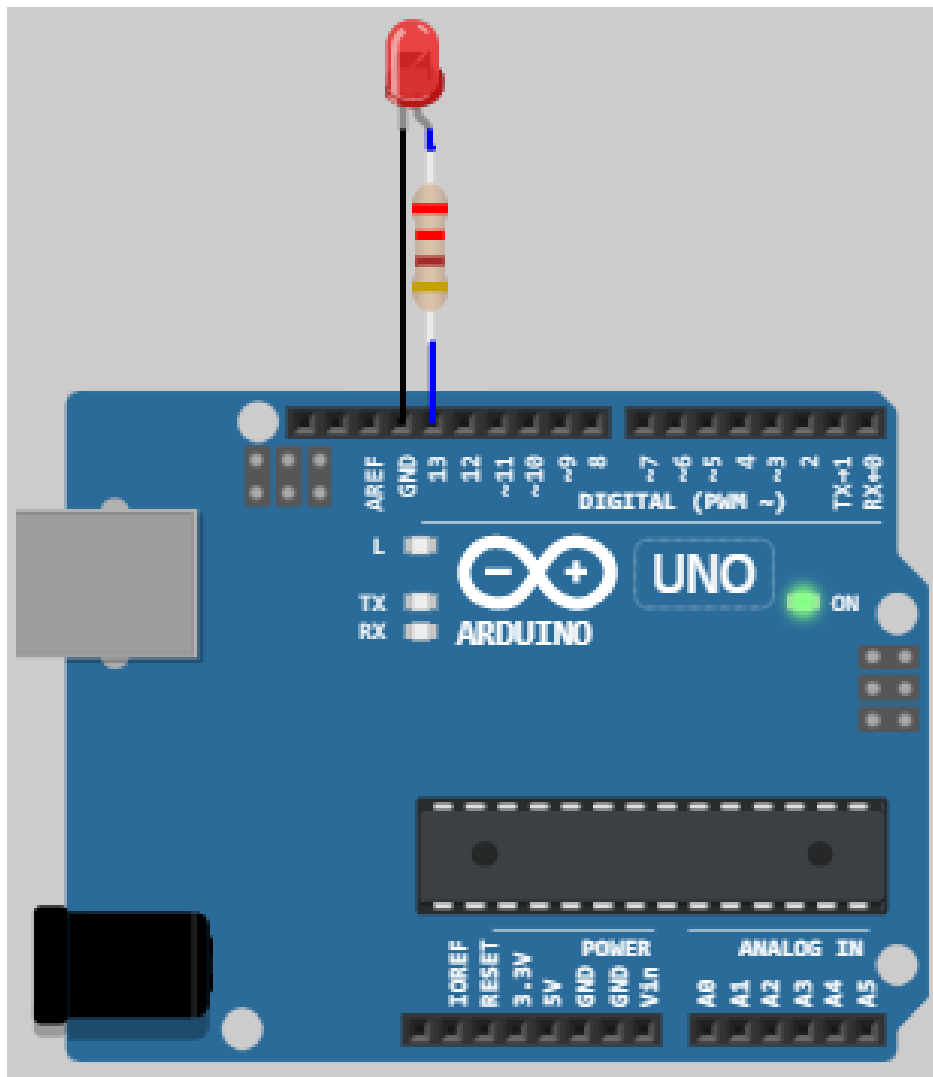


Figure 1.4 – Wokwi simulator running the Lab 4.1 circuit

1.6. System Architecture and Justification

The system follows a layered hardware-abstraction architecture divided into five tiers. The hardware tier consists of the physical GPIO pin, resistor, and LED. The MCAL (Microcontroller Abstraction Layer) tier is represented by the Led driver, which encapsulates all `pinMode()` and `digitalWrite()` calls behind a stable API. The ECAL (ECU Abstraction Layer) tier contains the `BinaryConditioner`, which implements the signal processing logic independent of any specific hardware. The service tier is represented by `ActuatorControl`, which combines the conditioner with the LED driver to form a self-contained actuator module. The application tier consists of the three FreeRTOS tasks and the shared state structure that orchestrate the overall system behaviour.

FreeRTOS is chosen over a simple cooperative scheduler because it provides true preemption and priority enforcement — the input reader task at priority 3 will always preempt the display task at priority 1, ensuring that serial input is never missed regardless of how long the display update takes. The mutex on the shared state structure prevents partially-written structs from being read by another task mid-update.

1.7. Case Study — Relevant Applicability of the Solution

Binary actuator control with signal conditioning is ubiquitous in industrial automation. A programmable logic controller (PLC) controlling a conveyor-belt drive motor, a solenoid valve in a pneumatic system, or a contactor in a motor control centre all employ debounce and persistence logic to prevent spurious state transitions from noisy digital inputs or momentary command spikes. The same pattern appears in building management systems where light relays and HVAC damper actuators must not cycle rapidly in response to transient occupancy sensor readings, since relay contact wear increases with switching frequency.

In consumer electronics, smart home relay modules use software debouncing to prevent a user's voice command from being applied multiple times due to repeated speech recognition events. In automotive body-control modules, door-lock actuators are controlled through validated discrete inputs to prevent uncommanded locking caused by electromagnetic interference on vehicle wiring harnesses. The laboratory's three-stage pipeline — saturation, debounce, persistence — directly mirrors the input processing found in safety-critical embedded systems described by the ISO 26262 and IEC 61508 functional-safety standards.

2. Design

2.1. System Architecture Diagrams

Component-Level Architecture

The system is composed of four hardware-connected components and one host interface, orchestrated by the Arduino Mega 2560. The serial terminal provides the user input path; the

LED on D7 is the binary actuator output; the I2C LCD provides continuous state feedback; and all components share a common ground and the 5 V supply from the board.

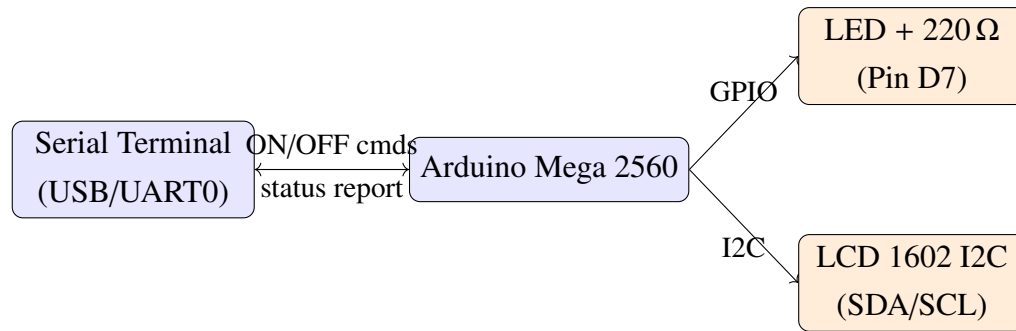


Figure 2.1 – System component-level architecture for Lab 4.1

Layered System Architecture

The software is organized into five layers following the automotive-inspired AUTOSAR-like architecture used throughout the lab series.

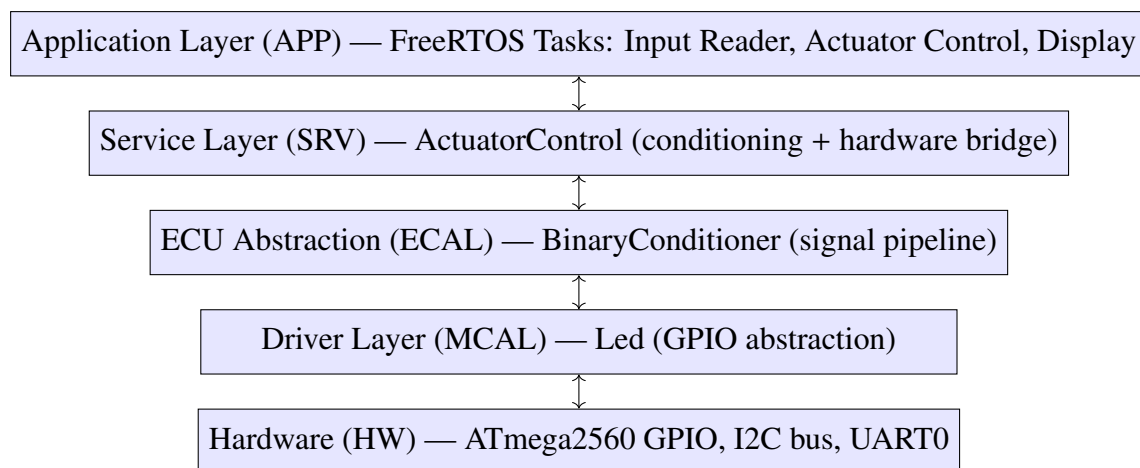


Figure 2.2 – Layered software architecture of the Lab 4.1 system

Each layer communicates only with the layer immediately above or below it through well-defined interfaces declared in header files. The `Led` driver isolates all `pinMode()` and `digitalWrite()` calls; the `BinaryConditioner` works purely on boolean values with no hardware knowledge; the `ActuatorControl` service wires those together; and the application tasks orchestrate timing and inter-task communication.

2.2. Block Diagrams

System Data Flow and FreeRTOS Task Architecture

The three FreeRTOS tasks communicate through a mutex-protected shared structure `g_actuator`. The arrows below show the direction of data flow between tasks.

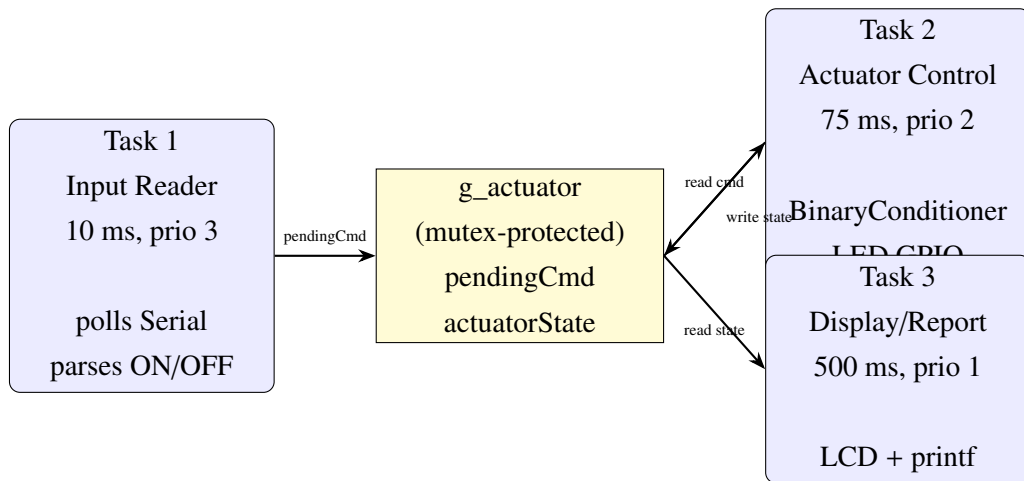


Figure 2.3 – FreeRTOS task data-flow diagram for Lab 4.1

BinaryConditioner Pipeline Flowchart

The `BinaryConditioner::process()` function runs on every Actuator Control task tick. It implements the three conditioning stages in sequence.

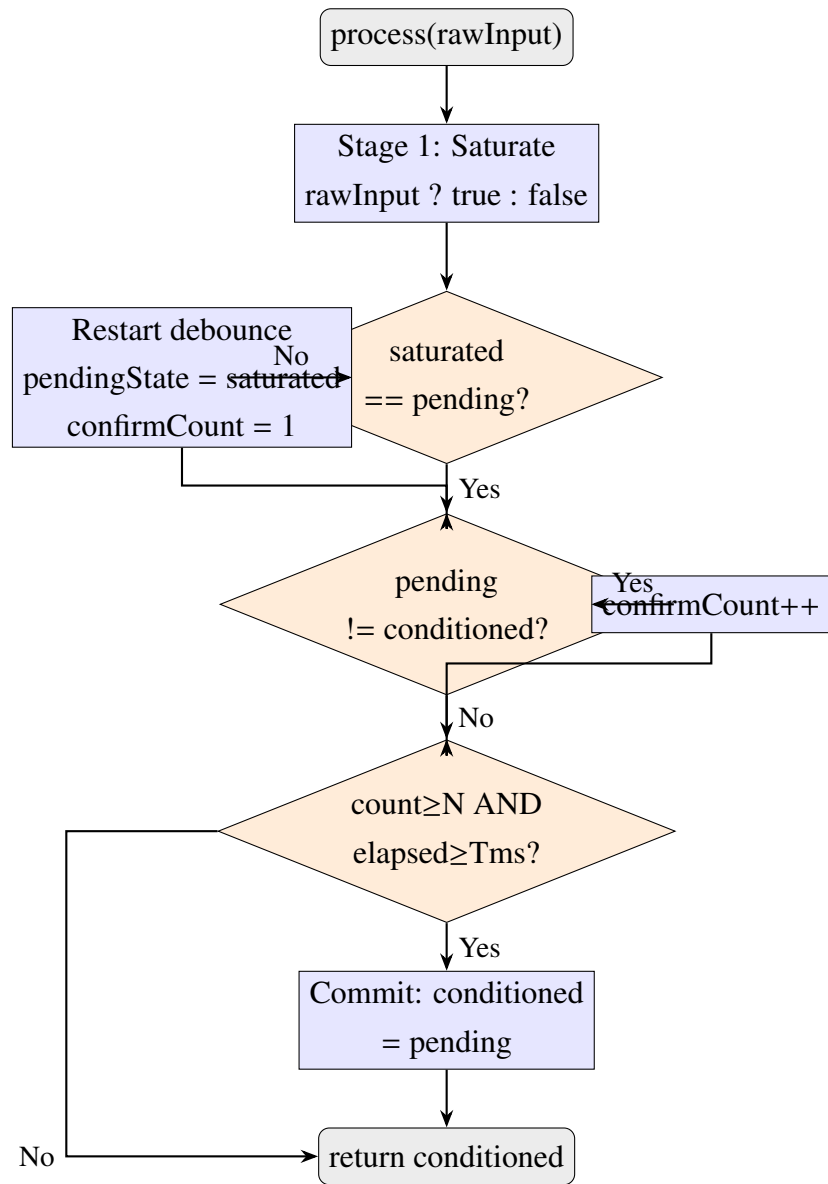


Figure 2.4 – Flowchart of `BinaryConditioner::process()`

Actuator Control Task Flowchart

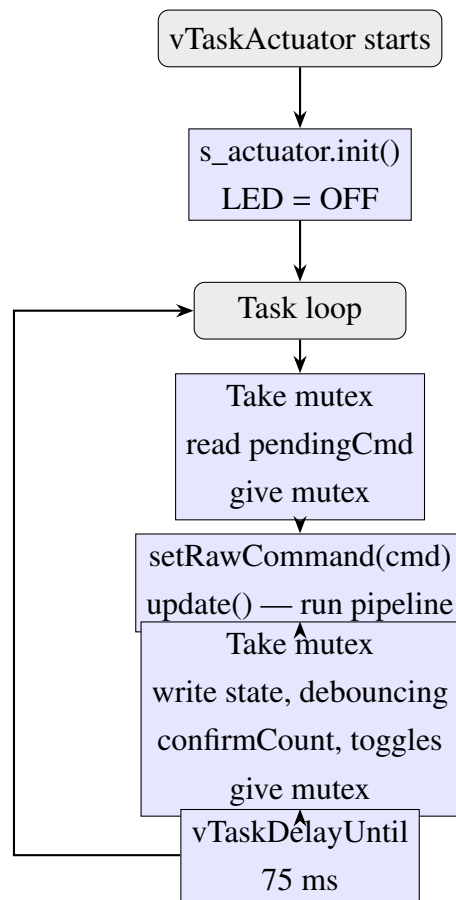


Figure 2.5 – Flowchart of the Actuator Control task (vTaskActuator)

2.3. Electrical Schematics

Component Specification

The circuit uses three components beyond the Arduino Mega 2560 itself. The red LED on pin D7 serves as the binary actuator output; its anode is connected to D7 through a series $220\ \Omega$ current-limiting resistor, and its cathode to GND. The 16x2 LCD module with PCF8574 I2C backpack is powered from the 5 V rail and communicates via the I2C bus on pins D20 (SDA) and D21 (SCL). The PCF8574 I2C address is 0x27. No pull-up resistors are required for the I2C lines as they are provided internally by the LCD backpack module.

Circuit Connections

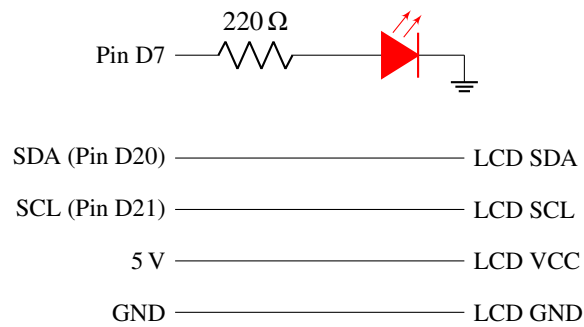


Figure 2.6 – Electrical schematic for Lab 4.1 (actuator LED and I2C LCD)

Hardware Configuration

The complete Wokwi simulation is defined in `labs/wokwi/lab4.1/diagram.json` and loaded via `wokwi.toml`. The configuration below lists all components and connections.

Listing 2.1 – Wokwi hardware configuration (`diagram.json`)

```
{
  "parts": [
    { "type": "wokwi-arduino-mega", "id": "mega" },
    { "type": "wokwi-lcd1602", "id": "lcd1", "attrs": { "pins": "i2c" } },
    { "type": "wokwi-led", "id": "led_act",
      "attrs": { "color": "red", "label": "Actuator" } },
    { "type": "wokwi-resistor", "id": "r_act",
      "attrs": { "resistance": "220" } }
  ],
  "connections": [
    ["lcd1:SDA", "mega:20", "cyan"],
    ["lcd1:SCL", "mega:21", "darkgreen"],
    ["mega:7", "r_act:1", "orange"],
    ["r_act:2", "led_act:A", "orange"],
    ["led_act:C", "mega:GND.2", "black"]
  ]
}
```

[Screenshot pending — run the Wokwi simulation and save to
resources/Design/ElectricalSchematics/wokwi_circuit.png]

Figure 2.7 – Wokwi simulation circuit (active run — LED lit, LCD showing state)

2.4. Project Structure

The `lab4_1` code is organized in the PlatformIO project under `labs/`, following the convention established in previous labs.

Listing 2.2 – Lab 4.1 project structure

```
labs/
|-- platformio.ini      (env:lab4_1 with lib_ldf_mode=deep+)
|-- src/main.cpp        (lab selector via #ifdef LAB4_1)
|-- lab/lab4_1/
|   |-- actuator_data.h/.cpp (shared state, timing constants, mutex)
|   |-- task_input.h/.cpp    (input reader task)
|   |-- task_actuator.h/.cpp (signal conditioning + hardware task)
|   |-- task_display.h/.cpp  (LCD + STDIO reporting task)
|   |-- lab4_1_main.h/.cpp   (entry point, task creation)
|-- lib/
|   |-- BinaryConditioner/   (3-stage binary signal conditioner)
|   |-- ActuatorControl/     (actuator + conditioner service)
|   |-- Led/                 (GPIO LED driver)
|   |-- LcdDisplay/          (I2C LCD driver)
|   |-- StdioSerial/         (STDIO -> UART redirection)
|-- wokwi/lab4.1/
|   |-- diagram.json         (circuit definition)
|   |-- wokwi.toml           (firmware path)
```

2.5. Modular Implementation

MCAL Layer: Led Driver

The Led driver provides a stable GPIO abstraction. It wraps a single pin identifier and tracks the current state in a bool field, so callers never need to use `digitalRead()` to query the state. The `init()` method configures the pin as an output and guarantees an initial OFF state, preventing the LED from flickering at boot.

Listing 2.3 – Led driver interface (Led.h excerpt)

```
class Led {
public:
    Led(uint8_t pin);
    void init();      // Set OUTPUT, initial OFF
    void turnOn();    // Set HIGH
    void turnOff();   // Set LOW
    void toggle();
    bool isOn() const;
private:
    uint8_t ledPin;
    bool    state;
};
```

ECAL Layer: BinaryConditioner

The BinaryConditioner implements the signal processing pipeline. It operates entirely on boolean values, making it hardware-independent. The process() method is called once per actuator task tick with the current raw command.

Listing 2.4 – BinaryConditioner key logic (BinaryConditioner.cpp)

```
bool BinaryConditioner::process(bool rawInput) {
    bool saturated = rawInput ? true : false; // Stage 1: Saturation
    _rawState = saturated;

    if (saturated != _pendingState) {
        // New value: restart validation
        _pendingState = saturated;
        _debounceStartMs = millis();
        _confirmCount = 1;
    } else if (_pendingState != _conditionedState) {
        _confirmCount++; // Stage 3: accumulate confirmations
    }

    if (_pendingState != _conditionedState) {
        _inDebounce = true;
        uint32_t elapsed = millis() - _debounceStartMs;
        // Stage 2+3: accept only when BOTH conditions satisfied
        if (_confirmCount >= _persistCount && elapsed >= _debouncePeriodMs) {
            _conditionedState = _pendingState;
            _inDebounce = false;
            _confirmCount = 0;
        }
    } else {
        _inDebounce = false;
    }
    return _conditionedState;
}
```

The key design choice is that stages 2 (debounce) and 3 (persistence) must both be satisfied simultaneously before a transition is committed. Requiring only the persistence count would fail against very rapid repeated commands; requiring only the debounce period would fail against periodic glitches spaced further apart than the window.

Service Layer: ActuatorControl

ActuatorControl is the service-layer module that wires the BinaryConditioner to the Led driver. Its update() method feeds the current raw command into the conditioner pipeline and drives the LED output only when the conditioned state changes. This avoids unnecessary digitalWrite() calls on every tick.

Listing 2.5 – *ActuatorControl::update() implementation*

```
void ActuatorControl::update() {
    bool rawBool      = (_rawCommand == State::ON);
    bool conditioned = _conditioner.process(rawBool);

    State newState = conditioned ? State::ON : State::OFF;
    if (newState != _currentState) {
        _currentState = newState;
        if (_currentState == State::ON) {
            _led.turnOn();
        } else {
            _led.turnOff();
        }
    }
}
```

Application Layer: FreeRTOS Tasks and Shared State

The three tasks communicate through `g_actuator`, a global `ActuatorShared` struct protected by a FreeRTOS mutex. Every read or write to the shared fields is surrounded by `xSemaphoreTake()` / `xSemaphoreGive()` calls with a 5–10 ms timeout. The short timeout ensures that if a task somehow stalls, other tasks are not blocked indefinitely.

Listing 2.6 – *Shared state structure (actuator_data.h)*

```
typedef struct {
    ActuatorControl::State pendingCmd;    // from input task
    ActuatorControl::State actuatorState; // from actuator task
    bool isDebouncing;
    uint8_t confirmCount;
    uint32_t toggleCount;
    SemaphoreHandle_t mutex;
} ActuatorShared;
```

The input reader task polls `Serial.available()` every 10 ms rather than blocking on `fgets()`. This non-blocking approach is essential in FreeRTOS: a blocking read would consume the task's CPU slice while waiting, preventing lower-priority tasks from running under some scheduler configurations. The 10 ms polling granularity is more than adequate for human-speed terminal input.

3. Obtained Results

3.1. Build Process

The Lab 4.1 firmware was compiled successfully using the PlatformIO `lab4_1` environment. The build system detected all seven library dependencies automatically via the `deep+` Li-

brary Dependency Finder mode: FreeRTOS, LiquidCrystal_I2C, ActuatorControl, BinaryConditioner, Led, StdioSerial, LcdDisplay, and the Arduino Wire library for I2C. The resulting firmware fits comfortably within the hardware limits of the ATmega2560: RAM usage is 18.9 % (1546 bytes out of 8192 bytes) and Flash usage is 7.1 % (18024 bytes out of 253952 bytes), leaving ample headroom for future extension.

[Screenshot pending — save to resources/ObtainedResults/build_results.png]
Must show the PlatformIO terminal output with [SUCCESS],
RAM/Flash usage percentages, and the dependency graph.

Figure 3.1 – Successful PlatformIO build output for Lab 4.1

3.2. Simulation Initialization

When the Wokwi simulation starts, the firmware prints the startup banner over the virtual serial terminal. The banner confirms the active environment, conditioning parameters (100 ms debounce, 3 confirmations), and task timing values. The LCD is initialized to display “Actuator Ctrl” on the first line and “Initializing...” briefly, then transitions to the main report loop. The initial state is OFF and the conditioning status is STABLE, which is correct since no command has been received yet.

[Screenshot pending — save to
resources/ObtainedResults/simulation_start.png]
Must show: Wokwi circuit + serial terminal with startup banner.
LED should be OFF. LCD should show “Actuator: OFF” / “STABLE”.

Figure 3.2 – Wokwi simulation initial state after startup banner

3.3. Command Interface Validation

Test: ON Command with Conditioning Delay

The first test verifies that typing ON in the serial terminal is correctly parsed, accepted, and eventually applied to the actuator after the conditioning pipeline is satisfied. Because the debounce window is 100 ms and the actuator task runs at 75 ms, the transition is committed after the second or third task tick following the command. During this interval the STDIO report shows “Cond: DEBOUNCING” and “Confirm: 1/3” or “2/3”. Once confirmed, the report shows “State: ON”, “Cond: STABLE”, and the LED turns on in the Wokwi simulation.

[Screenshot pending — save to
resources/ObtainedResults/test_on_command.png]
Type "ON" in terminal. Capture: terminal echo ">> Accepted: ON",
LED lighting up in Wokwi, LCD showing "Actuator: ON".

Figure 3.3 – Serial terminal after issuing the ON command

Test: OFF Command

After the actuator is ON, typing OFF causes the conditioning pipeline to enter the debouncing phase for the transition from ON to OFF. The same three-confirmation mechanism applies; the LED extinguishes and the LCD reverts to “Actuator: OFF” / “STABLE” after the required confirmations.

[Screenshot pending — save to
resources/ObtainedResults/test_off_command.png]
After ON is confirmed, type "OFF". Capture: LED off, LCD "Actuator:
OFF",
STDIO report showing State: OFF, Cond: STABLE.

Figure 3.4 – Serial terminal after issuing the OFF command

Test: Invalid Command Rejection

Entering an unrecognized string (e.g., BLINK or toggle) is handled gracefully. The input task prints an error message and reprints the prompt without modifying the pending command or the actuator state.

[Screenshot pending — save to
resources/ObtainedResults/test_invalid_command.png]
Type an invalid string (e.g. "BLINK"). Capture: error message "[ERR]
Unknown
command", actuator state unchanged, prompt reprinted.

Figure 3.5 – Invalid command error handling

Test: Periodic STDIO Report

Every four display ticks (approximately 2 seconds), the system prints a structured status report to the serial terminal. The report confirms all key system variables: the conditioned actuator state, the raw pending command, the conditioning status, the confirmation counter relative to the required count, and the cumulative toggle counter.

[Screenshot pending — save to resources/ObtainedResults/test_report.png]
Wait 2+ seconds after last command. Capture: STDIO report block showing
"–[Actuator Report]–" section with all fields.

Figure 3.6 – Periodic STDIO status report output

3.4. System Observations

The most notable observation during testing is the visible delay between issuing a command and the actuator responding. With a 100 ms debounce window and a 75 ms task period, the earliest

a state change can be committed is approximately 150–225 ms after the command is first received. This is intentional and demonstrates the conditioning pipeline working correctly. The delay would be imperceptible for a light relay application (where a sub-second response is acceptable) but would need to be reduced for time-critical control loops.

The LCD correctly reflects the conditioned state rather than the raw command. When the user types ON but the conditioning has not yet been satisfied, the LCD continues to show OFF with DEBOUNCING on the second line. This provides clear visual feedback that the system has received the request and is in the process of validating it. The STDIO report adds numerical detail by showing the confirmation count progress.

RAM usage of 1546 bytes (18.9 %) with three FreeRTOS tasks, two library instances, and a mutex leaves 6646 bytes free. This is consistent with the relatively small per-task stack sizes (192–320 words) chosen for this application. If additional complexity such as a keypad input or multiple actuators were added, stack sizes would need to be reviewed.

3.5. Verification Summary

- **Compilation:** Firmware compiles successfully for the lab4_1 environment. RAM 18.9 %, Flash 7.1 %.
- **Initialization:** Startup banner prints correctly; LCD and serial are both initialized before FreeRTOS tasks start.
- **Signal Conditioning:** All three stages (saturation, debounce, persistence) verified by observing the confirmation count and DEBOUNCING status during command transitions.
- **Command Parsing:** ON and OFF commands accepted; unknown input rejected with error message; backspace editing works correctly in the terminal.
- **Display:** LCD line 1 tracks conditioned state; line 2 tracks conditioning status. Periodic STDIO reports contain all required fields.
- **Architecture:** All hardware calls confined to library modules; application tasks contain no direct `digitalWrite()` or `analogRead()` calls.

4. Conclusion

4.1. Achievement of Objectives

The laboratory work successfully delivered all three required functional areas. The command interface correctly accepts ON and OFF strings from the serial terminal, echoes them back for confirmation, and rejects unrecognized input without disrupting the system state. The signal conditioning pipeline — implemented as a standalone reusable `BinaryConditioner` library — applies saturation, debouncing, and persistent state validation in sequence, ensuring that only stable and repeatedly confirmed commands reach the hardware output. The LCD display and STDIO

periodic report provide continuous feedback about both the conditioned actuator state and the current conditioning phase (stable or debouncing), meeting the display and reporting requirement at the 500 ms recurrence.

The three-task FreeRTOS architecture cleanly separates the input parsing, signal conditioning, and reporting concerns. Each task runs at an independently configured period and communicates exclusively through a mutex-protected shared structure, demonstrating the correct use of preemptive multitasking for a multi-rate embedded control system.

4.2. Performance Analysis and System Limitations

The main performance characteristic of the system is the introduced conditioning latency. A command is applied to the hardware after a minimum of approximately two actuator task periods (150 ms) plus the debounce window (100 ms), for a total worst-case delay of roughly 225 ms. This is acceptable for a lighting relay application but would be too slow for latency-sensitive control scenarios. The parameters are configurable via the constants in `actuator_data.h`; reducing `DEBOUNCE_PERIOD_MS` and `PERSIST_COUNT` trades noise rejection for lower latency.

A current limitation is that the input parser accepts commands only via UART serial. The requirement mentions keypad input as an alternative; this would require integrating the existing `KeypadInput` library and a command-mapping layer, but the signal conditioning and actuator control modules would remain unchanged since they operate on abstract boolean values.

The `lib_ldf_mode = deep+` setting added for this lab causes PlatformIO to scan all files in the `lib/` directory when building, which adds a few milliseconds to the build time. This is necessary because `actuator_data.h` includes `ActuatorControl.h` transitively; the default chain mode does not follow includes through non-source header files.

4.3. Proposed Improvements

Several enhancements could extend the current system. First, adding keypad support would allow standalone control without a PC connection, using the `KeypadInput` library to map specific keys (e.g., `*` for ON, `#` for OFF). Second, exposing the debounce period and persistence count as runtime-configurable parameters via serial commands (e.g., `SET DEBOUNCE 200`) would allow tuning without recompiling. Third, adding a toggle command (`TOGGLE`) would complement ON and OFF for applications where a pushbutton-style interaction is more natural. Fourth, replacing the LED with a relay module driven via a transistor or MOSFET gate would extend the system to control real mains-voltage loads, requiring only a pin mapping change in `actuator_data.h`.

4.4. Reflections on the Learning Experience

The most instructive aspect of this lab was understanding why a purely software-based signal conditioning pipeline is necessary even for simple boolean inputs. It is tempting to assume that a typed serial command is inherently clean and requires no conditioning; in practice, accidental double-sends, copy-paste artifacts in the terminal, and serial buffer overflows all produce spurious

command events that must be filtered. The persistent-validation stage provides a particularly elegant solution: because it requires the input to be confirmed multiple times, it naturally handles the case of a burst of identical commands without relying on wall-clock debouncing alone.

Working with multiple FreeRTOS tasks and a shared mutex also reinforced the importance of keeping critical sections as short as possible. Each task holds the mutex only for the time needed to read or write the specific fields it requires, which minimises the chance of priority inversion.

4.5. Impact of Technology in Real-World Applications

Binary actuator control with software signal conditioning is one of the most widely deployed embedded systems patterns across industries. In industrial automation, PLCs use identical debounce and filter logic on all digital input channels to prevent relay chatter and electromagnetic interference from causing false transitions in machine-state controllers. Smart home gateways apply persistence-based validation to occupancy sensor inputs before actuating HVAC dampers or lighting circuits, reducing energy waste from flickering states. In medical devices, binary outputs such as pump enable signals are validated through persistence counting to ensure that a single-bit memory error or bus glitch cannot inadvertently start or stop a critical drug delivery mechanism. The combination of a clearly defined signal processing pipeline with separation of hardware and application logic — as demonstrated in this laboratory — is a foundational design principle for all these safety-relevant systems.

5. Note Regarding Usage of AI

During the preparation of this laboratory report, the author utilized artificial intelligence tools to enhance the quality and presentation of the documentation. Specifically, the following AI tools were employed:

- **Report Generation:** Claude Code (Anthropic) was used to generate the full LaTeX report structure, all section content, TikZ diagrams, and CircuiTikZ schematics based on the implemented code and architecture.
- **Code Implementation:** Claude Code assisted in generating the `BinaryConditioner` and `ActuatorControl` library modules, the `lab4_1` FreeRTOS task files, and the Wokwi simulation configuration.
- **Report Formatting:** Assistance with LaTeX document structure, formatting consistency, and adherence to the established laboratory report template.
- **Code Documentation:** Generation of inline Doxygen-style comments within the source code to improve readability and maintainability.

All AI-generated content was thoroughly reviewed, validated, and adjusted by the author to ensure technical accuracy, alignment with project requirements, and adherence to embedded

systems best practices. The core technical architecture decisions, signal conditioning pipeline design, FreeRTOS task structure, and system integration represent the author’s understanding of the subject matter. The AI tools served as assistive technologies to enhance documentation quality and accelerate the development workflow, while the author retained full responsibility for the content, correctness, and conclusions presented in this report.

6. Bibliography

- [1] Bragarenco, A., AstaŃi, V., *Sisteme Electronice Ȃncorporate: Indicaȃii metodice pentru lucrări de laborator*, Universitatea Tehnică a Moldovei, Chişinău, 2024.
- [2] Technical University of Moldova, *Embedded Systems: Practical Guide for Hardware-Software Interface Development*, Department of Microelectronics and Biomedical Engineering, Chişinău, 2024.
- [3] FreeRTOS, *FreeRTOS Real Time Kernel — Official Documentation*, https://www.freertos.org/Documentation/RTOS_book.html, Accessed: 2026-03-28.
- [4] Arduino, *Arduino Mega 2560 Rev3 Documentation*, <https://docs.arduino.cc/hardware/mega-2560/>, Accessed: 2026-03-28.
- [5] PlatformIO, *Library Dependency Finder (LDF) Documentation*, <https://docs.platformio.org/en/latest/librarymanager/ldf.html>, Accessed: 2026-03-28.
- [6] Wokwi, *Online Electronics Simulator — Arduino, ESP32, Raspberry Pi Pico*, <https://wokwi.com/>, Accessed: 2026-03-28.
- [7] International Electrotechnical Commission, *IEC 61508: Functional Safety of E/E/PE Safety-related Systems*, IEC, Geneva, 2010.

7. Appendix

The complete source code is organized below by architectural layer. All source files are available in the project GitHub repository.

GitHub Repository: <https://github.com/mcittkmims/es-labs>

7.1. Application Layer

Main Entry Point

Listing 7.1 – src/main.cpp — lab selector (Lab 4.1 excerpt)

```
// Automatically select the active lab based on build flags
#elif defined(LAB4_1)
    #include "lab4_1_main.h"
```

```

void setup() {
#ifdef LAB4_1
    lab4_1Setup();
}
void loop() {
#ifdef LAB4_1
    lab4_1Loop();
}

```

Lab 4.1 Entry Point

Listing 7.2 – lab4_1/lab4_1_main.h

```

void lab4_1Setup();    // Initialize hardware, shared state, and FreeRTOS tasks
void lab4_1Loop();     // Empty - all logic in FreeRTOS tasks

```

Listing 7.3 – lab4_1/lab4_1_main.cpp

```

#include "lab4_1_main.h"
#include "actuator_data.h"
#include "task_input.h"
#include "task_actuator.h"
#include "task_display.h"
#include <Arduino_FreeRTOS.h>
#include "StdioSerial.h"

void lab4_1Setup() {
    stdioSerialInit(9600);
    printf("Lab 4.1 -- Binary Actuator Control\r\n");
    printf("Debounce: %u ms, Persist: %u confirms\r\n",
        DEBOUNCE_PERIOD_MS, PERSIST_COUNT);

    actuatorDataInit();

    xTaskCreate(vTaskInputReader, "Input", TASK_INPUT_STACK,
        NULL, TASK_INPUT_PRIORITY, NULL);
    xTaskCreate(vTaskActuator, "Actuator", TASK_ACTUATOR_STACK,
        NULL, TASK_ACTUATOR_PRIORITY, NULL);
    xTaskCreate(vTaskDisplay, "Display", TASK_DISPLAY_STACK,
        NULL, TASK_DISPLAY_PRIORITY, NULL);
}

void lab4_1Loop() { /* All logic in FreeRTOS tasks */ }

```

Shared State and Constants

Listing 7.4 – lab4_1/actuator_data.h

```
#define TASK_INPUT_PERIOD_MS      10U
#define TASK_ACTUATOR_PERIOD_MS  75U
#define TASK_DISPLAY_PERIOD_MS   500U
#define STDIO_REPORT_EVERY_N     4U
#define PIN_ACTUATOR_LED         7U
#define DEBOUNCE_PERIOD_MS       100U
#define PERSIST_COUNT            3U

typedef struct {
    ActuatorControl::State pendingCmd;
    ActuatorControl::State actuatorState;
    bool                    isDebouncing;
    uint8_t                 confirmCount;
    uint32_t                toggleCount;
    SemaphoreHandle_t       mutex;
} ActuatorShared;

extern ActuatorShared g_actuator;
void actuatorDataInit();
```

Input Reader Task

Listing 7.5 – lab4_1/task_input.cpp

```
void vTaskInputReader(void *params) {
    char buf[32]; uint8_t len = 0;
    printf("Commands: ON | OFF\r\n> ");
    TickType_t lastWake = xTaskGetTickCount();
    while (true) {
        while (Serial.available()) {
            char c = (char)Serial.read();
            Serial.write(c);
            if (c == '\r' || c == '\n') {
                if (len > 0) {
                    buf[len] = '\0'; normalizeInput(buf);
                    if (strcmp(buf, "ON") == 0) {
                        // take mutex, write pendingCmd = ON, give mutex
                    } else if (strcmp(buf, "OFF") == 0) {
                        // take mutex, write pendingCmd = OFF, give mutex
                    } else {
                        printf(" [ERR] Unknown: '%s'\r\n", buf);
                    }
                    len = 0; printf("> ");
                }
                } else if (len < sizeof(buf)-1) buf[len++] = c;
        }
        vTaskDelayUntil(&lastWake, pdMS_TO_TICKS(TASK_INPUT_PERIOD_MS));
    }
}
```

```

    }
}

```

Actuator Control Task

Listing 7.6 – lab4_1/task_actuator.cpp

```

static ActuatorControl s_actuator(PIN_ACTUATOR_LED,
                                   DEBOUNCE_PERIOD_MS, PERSIST_COUNT);

void vTaskActuator(void *params) {
    s_actuator.init();
    TickType_t lastWake = xTaskGetTickCount();
    while (true) {
        // 1. Read raw command
        ActuatorControl::State rawCmd;
        xSemaphoreTake(g_actuator.mutex, pdMS_TO_TICKS(5));
        rawCmd = g_actuator.pendingCmd;
        xSemaphoreGive(g_actuator.mutex);

        // 2. Run signal conditioning pipeline + drive GPIO
        s_actuator.setRawCommand(rawCmd);
        s_actuator.update();

        // 3. Write conditioned state back to shared memory
        xSemaphoreTake(g_actuator.mutex, pdMS_TO_TICKS(5));
        if (s_actuator.getState() != g_actuator.actuatorState)
            g_actuator.toggleCount++;
        g_actuator.actuatorState = s_actuator.getState();
        g_actuator.isDebouncing = s_actuator.isDebouncing();
        g_actuator.confirmCount = s_actuator.getConfirmCount();
        xSemaphoreGive(g_actuator.mutex);

        vTaskDelayUntil(&lastWake, pdMS_TO_TICKS(TASK_ACTUATOR_PERIOD_MS));
    }
}

```

Display and Reporting Task

Listing 7.7 – lab4_1/task_display.cpp

```

static LcdDisplay s_lcd(0x27, 16, 2);

void vTaskDisplay(void *params) {
    s_lcd.init();
    s_lcd.showTwoLines("Actuator Ctrl", "Initializing...");
    uint8_t tick = 0;
    TickType_t lastWake = xTaskGetTickCount();
}

```



```

while (true) {
    // Read shared state, update LCD, print STDIO report every 4 ticks
    // LCD line 1: "Actuator: ON " or "Actuator: OFF"
    // LCD line 2: "STABLE    " or "DEBOUNCING"
    vTaskDelayUntil(&lastWake, pdMS_TO_TICKS(TASK_DISPLAY_PERIOD_MS));
}
}

```

7.2. Service Layer

ActuatorControl

Listing 7.8 – lib/ActuatorControl/ActuatorControl.h

```

class ActuatorControl {
public:
    enum class State { OFF, ON };
    ActuatorControl(uint8_t pin, uint32_t debouncePeriodMs=100,
                    uint8_t persistCount=3);
    void init();
    void setRawCommand(State cmd);
    void update();           // run pipeline + drive GPIO
    State getState() const;  // conditioned hardware state
    State getRawCommand() const;
    bool isDebouncing() const;
    uint8_t getConfirmCount() const;
    uint8_t getRequiredConfirmCount() const;
private:
    Led _led;
    BinaryConditioner _conditioner;
    State _rawCommand;
    State _currentState;
};

```

Listing 7.9 – lib/ActuatorControl/ActuatorControl.cpp (key method)

```

void ActuatorControl::update() {
    bool rawBool    = (_rawCommand == State::ON);
    bool conditioned = _conditioner.process(rawBool);
    State newState  = conditioned ? State::ON : State::OFF;
    if (newState != _currentState) {
        _currentState = newState;
        if (_currentState == State::ON) _led.turnOn();
        else _led.turnOff();
    }
}

```

7.3. ECAL Layer

BinaryConditioner

Listing 7.10 – lib/BinaryConditioner/BinaryConditioner.h

```
class BinaryConditioner {
public:
    BinaryConditioner(uint32_t debouncePeriodMs=100, uint8_t persistCount=3);
    bool    process(bool rawInput);    // run 3-stage pipeline
    bool    getConditionedState() const;
    bool    getRawState()             const;
    bool    getPendingState()         const;
    bool    isDebouncing()            const;
    uint8_t getConfirmCount()         const;
    uint8_t getRequiredConfirmCount() const;
    void    reset();
private:
    uint32_t _debouncePeriodMs;
    uint8_t  _persistCount;
    bool     _rawState, _pendingState, _conditionedState;
    uint32_t _debounceStartMs;
    bool     _inDebounce;
    uint8_t  _confirmCount;
};
```

Listing 7.11 – lib/BinaryConditioner/BinaryConditioner.cpp

```
bool BinaryConditioner::process(bool rawInput) {
    bool saturated = rawInput ? true : false;    // Stage 1: Saturation
    _rawState = saturated;

    if (saturated != _pendingState) {
        _pendingState = saturated;                // restart on change
        _debounceStartMs = millis();
        _confirmCount = 1;
    } else if (_pendingState != _conditionedState) {
        _confirmCount++;                          // Stage 3: accumulate
    }

    if (_pendingState != _conditionedState) {
        _inDebounce = true;
        uint32_t elapsed = millis() - _debounceStartMs;
        // Stage 2+3: commit only when BOTH satisfied
        if (_confirmCount >= _persistCount && elapsed >= _debouncePeriodMs) {
            _conditionedState = _pendingState;
            _inDebounce = false; _confirmCount = 0;
        }
    } else { _inDebounce = false; }
```

```
    return _conditionedState;
}
```

7.4. MCAL Layer

Led Driver

Listing 7.12 – lib/Led/Led.h (interface summary)

```
class Led {
public:
    Led(uint8_t pin);
    void init();           // pinMode OUTPUT, initial OFF
    void turnOn();         // digitalWrite HIGH
    void turnOff();        // digitalWrite LOW
    void toggle();
    bool isOn() const;
private:
    uint8_t ledPin;
    bool    state;
};
```

7.5. PlatformIO Configuration

Listing 7.13 – platformio.ini — Lab 4.1 environment

```
[env:lab4_1]
platform = atmelavr
board = megaatmega2560
framework = arduino
monitor_speed = 9600
build_src_filter = +<*> +<../lab/lab4_1/*>
build_flags = -I lab/lab4_1 -DLAB4_1
lib_ldf_mode = deep+
lib_deps =
    feilipu/FreeRTOS
    marcoschwartz/LiquidCrystal_I2C@^1.1.4
```